Abstract Data Types

- An **abstract data type** (ADT) is a conceptual description of data and operations defined on that data.

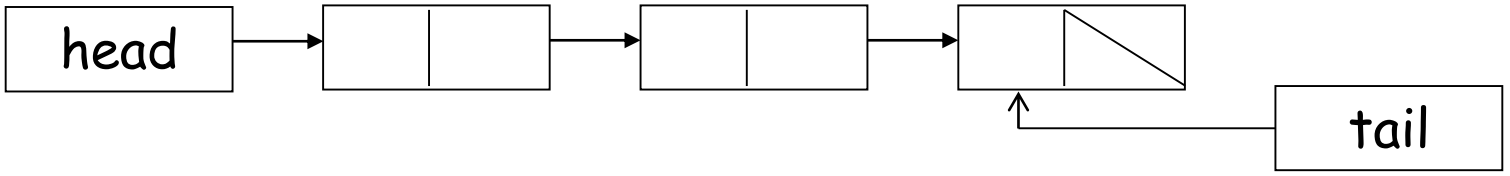
- Examples:
  - **int**: it is the set of integers (up to a certain magnitude), with operations +, -, /, *
  - **stack**: it is the set of objects (defined on its domain), with operations *push, pop*.

Data | Interface | Implementation

List

Storing data items in arrays has at least **two** limitations:

- The array size is fixed once it is created: Changing the size of the array requires creating a new array and then copying all data from the old array to the new array.
- The data items in the array are next to each other in memory: Inserting an item inside the array requires shifting other items.

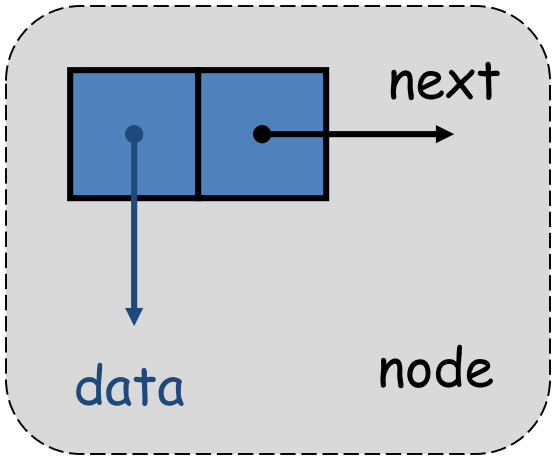


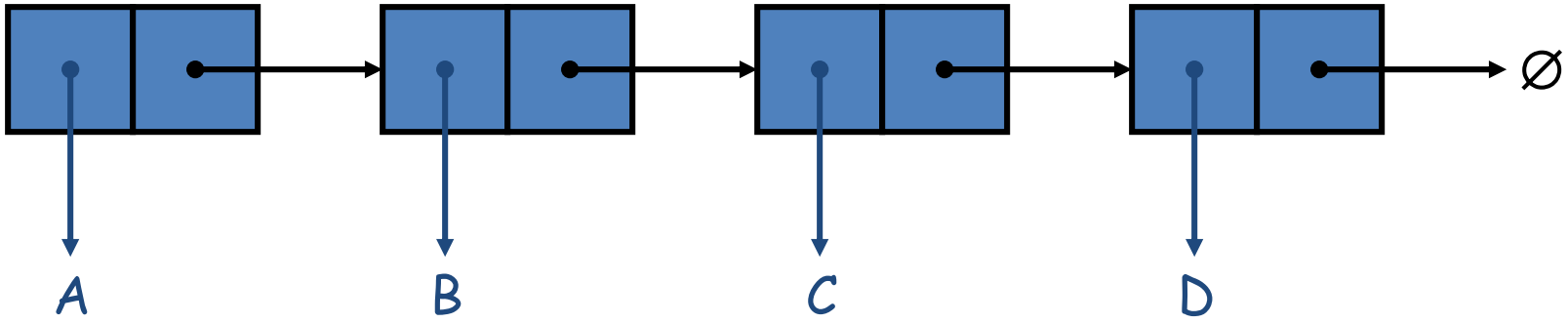
List ADT

- The List ADT models a sequence of positions (p) storing arbitrary objects (o).
- It allows for insertion and deletion in the "middle".
- Query methods:
  - isFirst(p), isLast(p)
- Access methods:
  - first(), last()
  - before(p), after(p)
- Update methods:
  - replaceElement(p, o)
  - swapElements(p, q)
  - insertBefore(p, o)
  - insertAfter(p, o)
  - insertFirst(o)
  - insertLast(o)
  - remove(p)

Singly Linked List

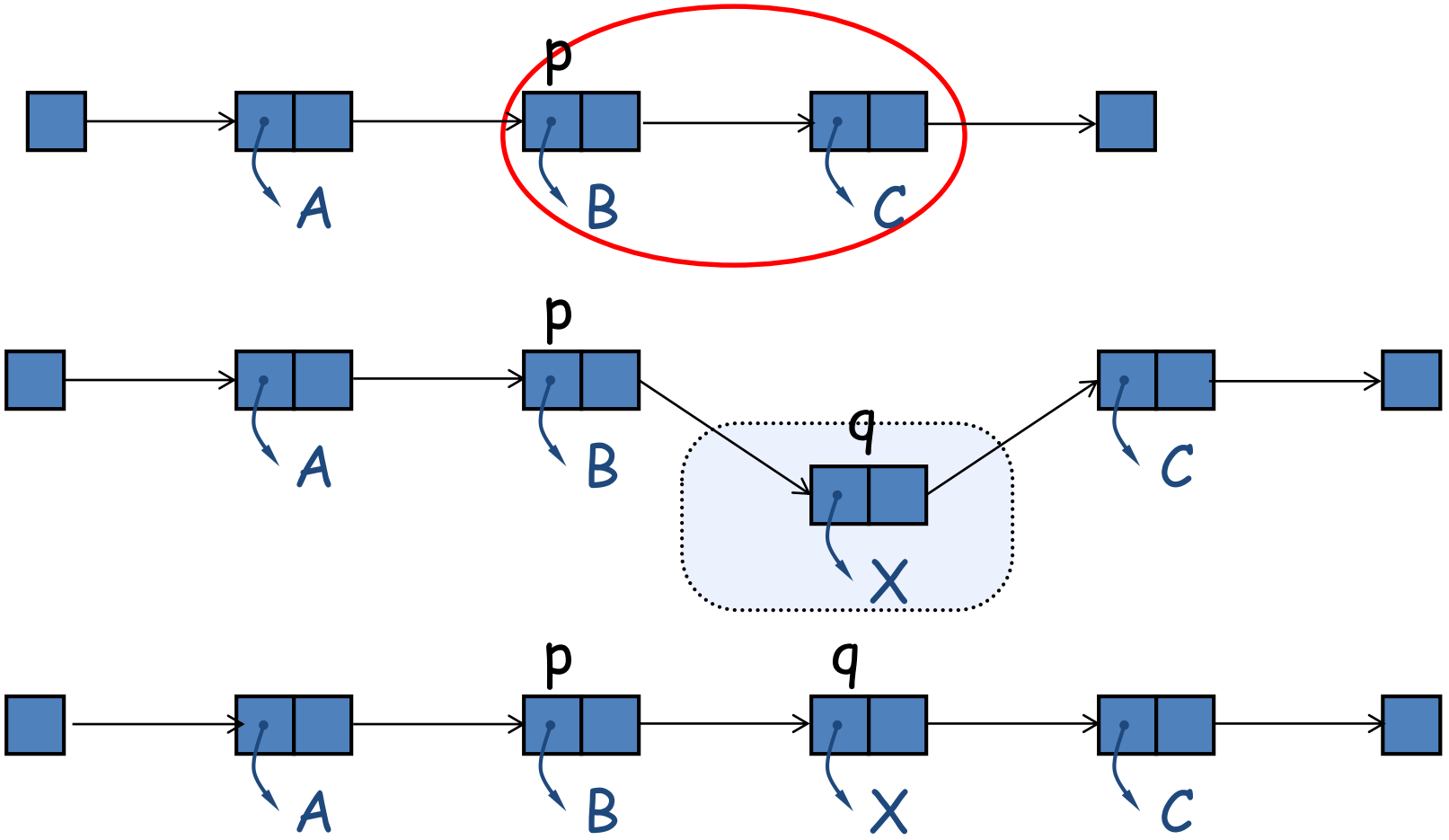
- A singly linked list is a data structure consisting of a sequence of nodes.
- Each node stores
  - **data**
  - **link** to the next node





Insertion (concept)

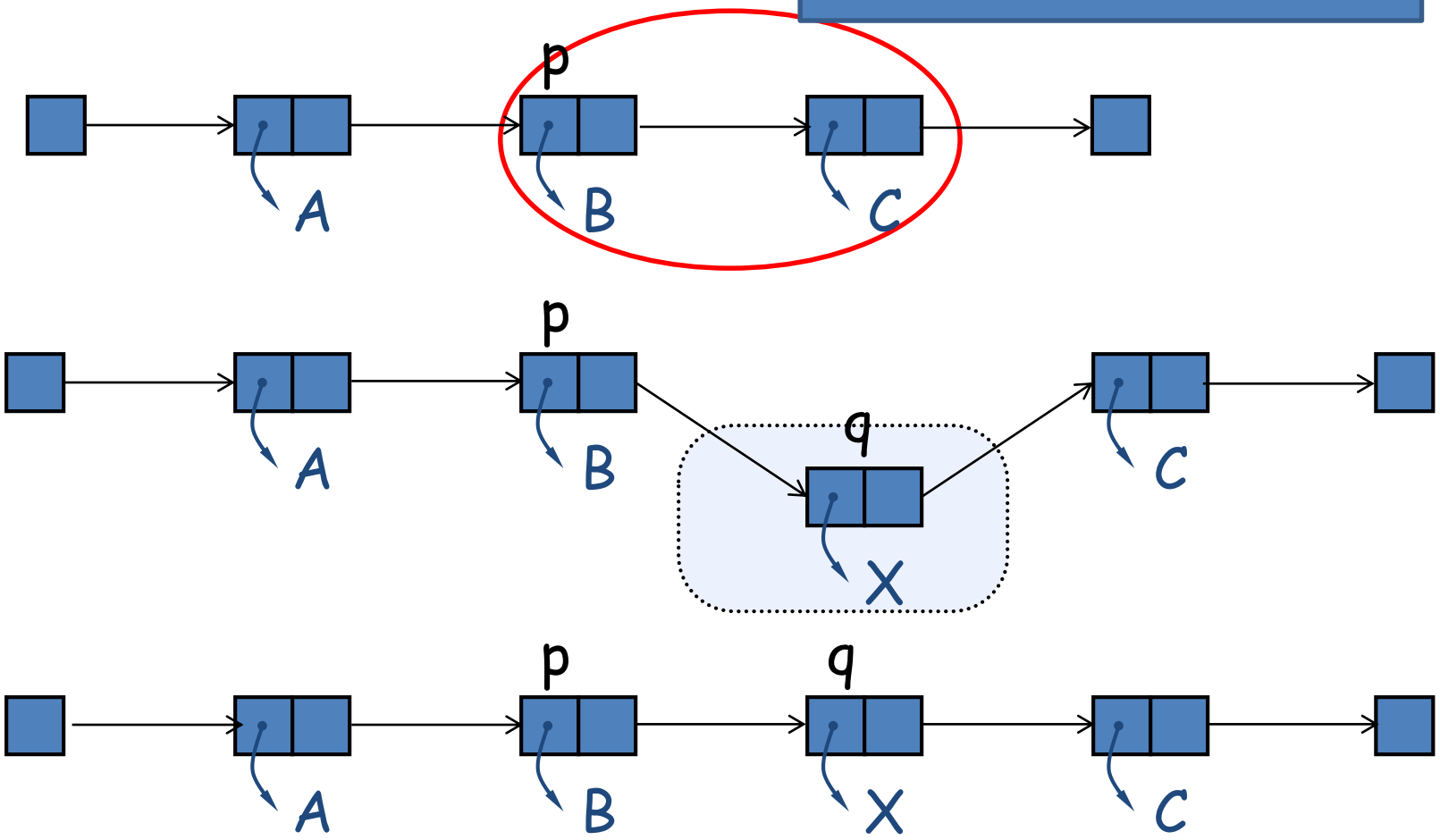
We visualize operation insertAfter(p, X), which returns position q.



Insertion (operation)

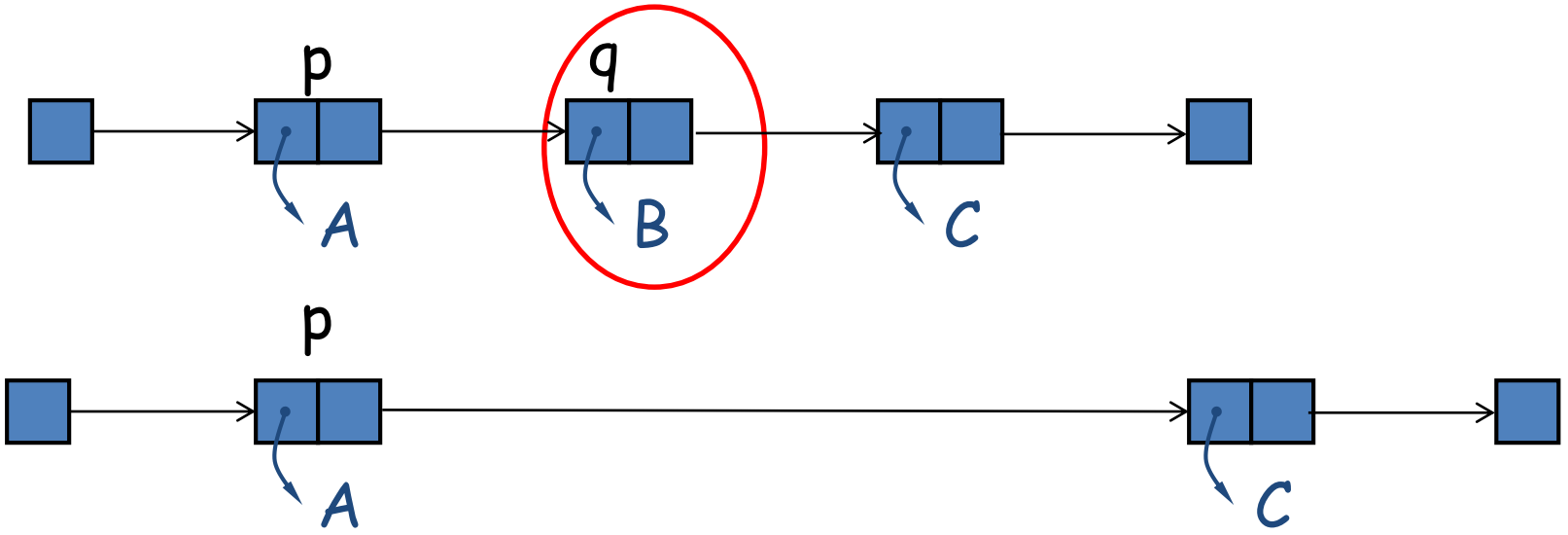
We visualize operation insertAfter(p, X), which returns position q.

```
q.data = X;
q.next = p.next;
P.next = q;
```



Deletion (concept)

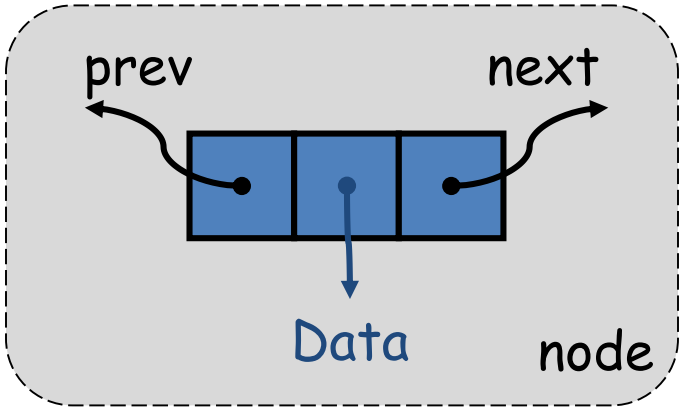
Let us apply delete operation remove(q)

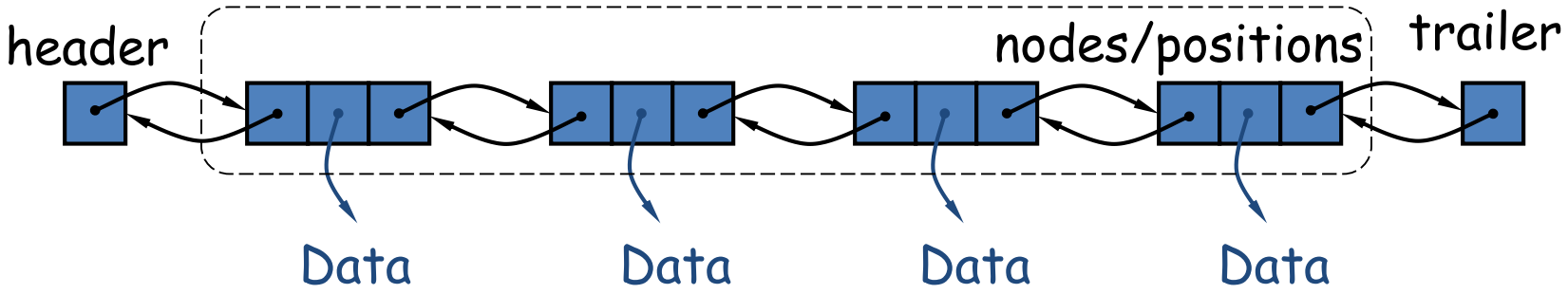



```
//find the previous node p
p.next = q.next;
delete q;
```

Doubly Linked List

- A doubly linked list provides a natural implementation of the List ADT.
- Nodes implement Position and store:
  - Data
  - link to the previous node
  - link to the next node
- Special trailer and header nodes

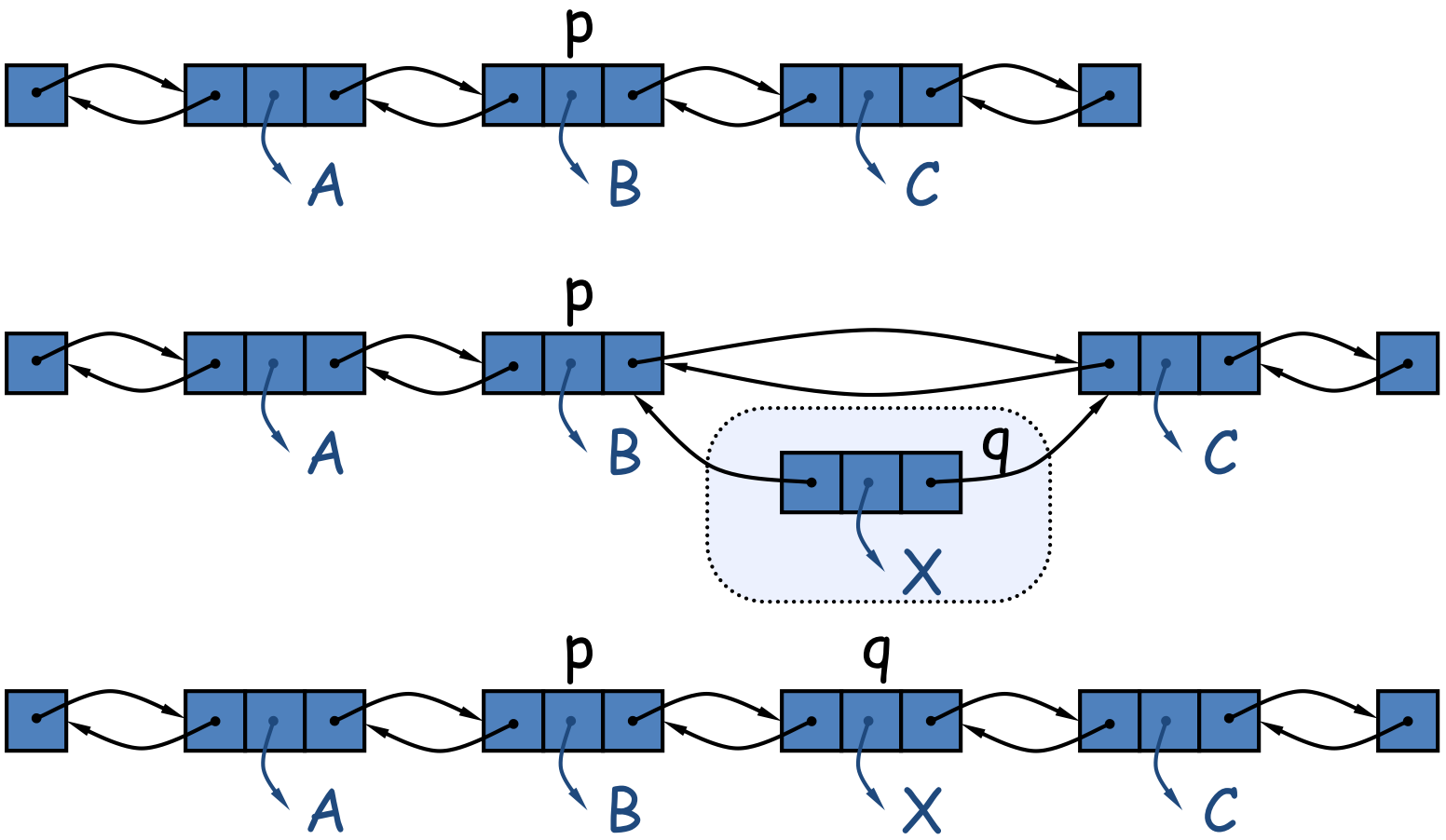




Insertion

We visualize operation insertAfter(p, X), which returns position q.

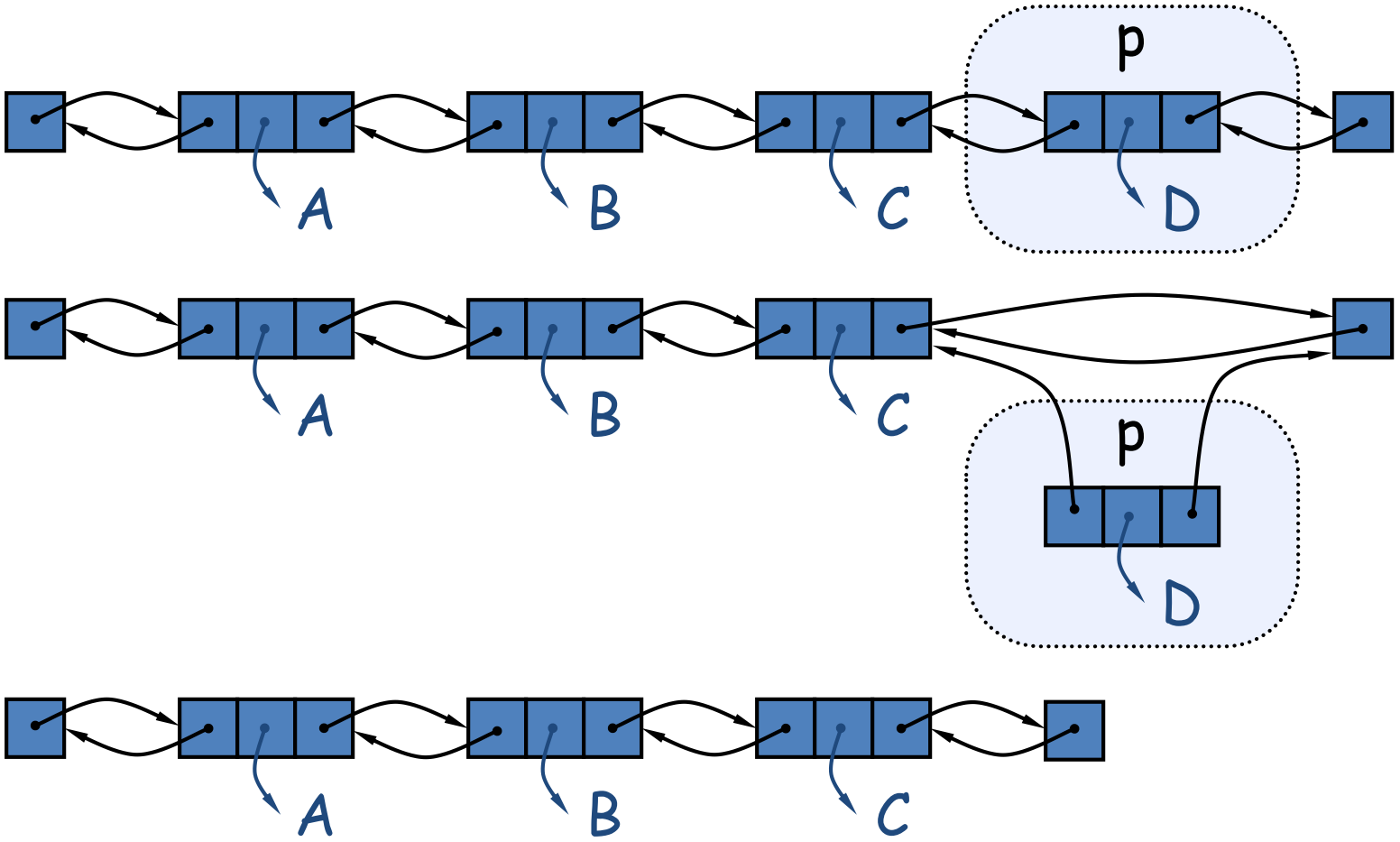
```
q.data = X;
q.prev = p;
q.next = p.next;
p.next = q;
(p.next).prev = q;
//(q.next).prev = q;
```



Deletion

```
(p.prev).next = p.next;
(p.next).prev = p.prev;
delete p;
```

We visualize remove(p), where p = last()



Performance

In the implementation of the List ADT by means of a doubly linked list

- The space used by a list with n elements is $O(n)$.
- The space used by each position of the list is $O(1)$.
- All the operations of the List ADT run in $O(1)$ time.
- Operation find() runs in $O(n)$ time in worst case.